

Document number: P0756R0

Date: 2017-08-12

Project: ISO JTC1/SC22/WG21, Programming Language C++

Audience: Evolution Working Group

Reply to: Arthur O'Dwyer <arthur.j.odwyer@gmail.com>

Lambda syntax should be more liberal in what it accepts

[1. Introduction](#)

[2. Motivations](#)

[3. "Simple. The default goes first."](#)

[4. Proposed wording](#)

[8.1.5.2 Captures \[expr.prim.lambda.capture\]](#)

[5. References](#)

1. Introduction

This paper proposes to remove two minor stumbling blocks to the teachability of lambda syntax — stumbling blocks of which many programmers may not even be aware.

The first is that `[&x, =]() { return x; }` is ill-formed.

The second is that `[=, y]() { return x; }` is ill-formed.

I propose to make both of these lambdas well-formed, with the obvious meanings.

This proposal is effectively a superset of Thomas Köppe's recent proposal "Allow lambda capture `[=, this]`" [\[P0409R2\]](#); the difference is that P0409 proposes to add a single word to `[expr.prim.lambda.capture]` whereas this paper proposes to strike the entire offending *sentence* from that clause.

2. Motivations

Here is an excerpt from a tutorial I have been writing on lambdas in C++17:

One of the ways in which lambda-expressions add expressive power to the language (that is, one way in which they're more than just syntactic sugar for the C++03 equivalents that we've been showing so far in this chapter) is that you can take the tedious work of counting the captures you need, and foist that work off on the compiler. To capture "everything needed by this lambda, and nothing else," simply add a single `=` to your capture-list (to capture by value), or add a single `&` (to capture by reference).

This "capture-default" syntax can be prepended onto the other syntaxes previously discussed, so that for example `[=, p=q, &r]` means *"my data member p should be initialized to a copy of q from the enclosing scope; my data member r should be a reference to the r in the enclosing scope; and anything else I need should be captured by value."*

Unfortunately, the syntax is not as forgiving as you might expect; for example placing the `=` at the end of the list (where it logically belongs) is a syntax error, and "redundant" C++11-style captures are not allowed — for example `[=, x]` is forbidden but `[=, x=x]` is permitted.

The final paragraph above is unfortunate. It would be easier to teach C++, and less verbiage would be expended on syntactic trivia, if we loosened up the arbitrary requirements above.

Another motivation for loosening the prohibition against "redundant" captures is to improve our friendliness to machine-generated or mechanically-refactored code. The default capture modes (such as `[=]`) are useful in allowing us to write lambdas without manually counting the variables that need capturing. But if we have in mind one particular variable that *does* need capturing, regardless of what other variables are odr-used within the body of the lambda, it would be convenient to be able to specify it directly as a C++11-style capture. For example:

```
struct TokenPool : std::enable_shared_from_this<TokenPool> {
    TokenPool(int initial_number_of_tokens);
    std::shared_ptr<bool> get() {
        auto shared_this = shared_from_this();
        bool *p;
        // ... block until some "p" is available, then remove it from the pool ...
        return std::shared_ptr<bool>(p, [shared_this](bool *p){
            shared_from_this->putback(p);
        });
    }
private:
    void putback(bool *p) {
```

```

        // ... put "p" back in the pool ...
    }
};
ThreadPool token_pool(100);

struct Task : std::enable_shared_from_this<Task> {
    void run();
};

void Task::run()
{
    auto shared_this = shared_from_this();
    auto token = token_pool.get(); // block; don't launch our new thread until there's a token available for us
    for (int i=0; i < 100; ++i) {
        worker_pool.launch([token, shared_this, i]() {
            shared_this->execute_one_shard_of_task(i);
        });
        // We'll hold onto that token until every one of these lambdas has finished running.
        // Then the token will put itself back in the pool.
    }
}

```

The important piece of the above code sample is the lambda-expression whose lambda-capture is `[token, shared_this, i]`. Under the current standard, this is as far as we can simplify the code using just C++11-style captures. An alternative way to write it is `[=, token=token]() { ... }`, but that awkwardly duplicates the name of identifier `token`. (We could pick a shorter name for the left-hand side of that assignment — for example `[=, x=token]` — but then we'd have to verify manually that there was no use of any existing `x` inside the lambda's body.)

Under this proposal, the syntaxes `[=, token]` and `[token, =]` would both be supported for the above example's capture-list.

3. "Simple. The default goes first."

In discussion on the std-proposals mailing list [\[Bolas\]](#), the following objection was raised:

Seeing ``='` first makes it clear to the reader that this is a value default-capturing lambda. Seeing ``&`` first makes it clear that it's a reference default-capturing lambda. Seeing neither makes it clear that it captures only what is listed. It's a simple rule to remember: ***put the default first***. I don't think that rule affects teachability. [...] Unless you think that "default initialization" is somehow related to and should be syntactically consistent with "default parameters" or "default capture".

My rebuttal is that currently C++ supports "defaults" or "default behavior" in the following places associated with the *trailing end*:

- Default values for function arguments (such arguments can appear only at the *end*)
- C-style variadic "... " soaks up unmatched arguments (only at the *end*)
- Variadic template parameter packs soak up unmatched arguments (technically anywhere but typically you find them at the *end*)
- Aggregate initializer "set the rest of my fields to zero" (only for *trailing* fields)

And in the following places, "default behaviors" can appear in any order, intermixed with non-defaults:

- Default values for template parameters
- Default values for non-static data members

In only this one place does the "default behavior" for a C++ construct appear at the *beginning* of a construct:

- Default capturing behavior for lambda expressions (only at the *beginning*)

The simple rule *put the default first* is not a rule that appears anywhere else in C++; while many places in C++ encourage the student to learn the competing rule, ***put the default last***.

Normal English usage is also on the side of "default last": it feels natural to say

Put the apple in the basket, the banana in the bowl, and everything else in the cupboard.

It feels *unnatural* to say

Put everything else in the cupboard, the apple in the basket, and the banana in the bowl.

Yet the latter is what C++11 lambda syntax requires from the student!

This is not intended to convince you that the "right" place for the capture-default is in the trailing position. This is merely intended to convince you that many students naturally *try* (and even *want*) to put the default in the trailing position, and get frustrating compiler errors as a result. Here are the compiler error messages from GCC, Clang, and Visual C++ as of this writing:

```

(GCC 8.0.0)
prog.cc: In function 'int main()':
prog.cc:9:9: error: expected identifier before '=' token
    [x, =]() { };
        ^

(Clang 6.0.0)
prog.cc:9:9: error: expected variable name or 'this' in lambda capture list
    [x, =]() { };
        ^

(Visual C++ 19.00.23506)
source_file.cpp(9): error C2337: 'x': attribute not found
source_file.cpp(9): error C2059: syntax error: '='
source_file.cpp(9): error C2143: syntax error: missing ';' before '{'

```

Just changing all the major compilers to produce better error messages would be an improvement on the status quo. But even better would be to make the code compile on the first shot!

4. Proposed wording for C++20

The wording in this section is relative to WG21 draft N4659 [\[N4659\]](#), that is, the current draft of the C++17 standard as of July 2017.

8.1.5.2 Captures [expr.prim.lambda.capture]

Edit the grammar in paragraph 1 as follows.

```

lambda-capture:
  capture-default
  capture-list
  capture-default , capture-list
capture-default:
  &
  =
capture-list:
  capture-default
  capture-list , capture-default
  capture ...opt
  capture-list , capture ...opt
capture:
  simple-capture
  init-capture
simple-capture:
  identifier
  & identifier
  this
  * this
init-capture:
  identifier initializer
  & identifier initializer

```

Edit paragraph 2 as follows.

~~If a lambda-capture includes a capture-default that is &, no identifier in a simple-capture of that lambda-capture shall be preceded by &. If a lambda-capture includes a capture-default that is =, each simple-capture of that lambda-capture shall be of the form “& identifier” or “* this”. [Note: The form [&, this] is redundant but accepted for compatibility with ISO C++ 2014. — end note]~~ A lambda-capture shall contain no more than one capture-default. Ignoring appearances in initializers of init-captures, an identifier or this shall not appear more than once in a lambda-capture. [Example:

```

struct S2 { void f(int i); };
void S2::f(int i) {
    [&, i]{ };           // OK
    [&, &i]{ };          // OK
     [&, i, &]{ };      // error: more than one capture-default
    [=, *this]{ };       // OK
    [=, this]{ };        // OK
    [i, i]{ };           // error: i repeated

```

```
.[i, &i]{ }; // error: i repeated  
[this, *this]{ }; // error: this appears twice  
}  
  
— end example]
```

5. References

[Bolas]

std-proposals forum thread "P0756R0: Lambda syntax should be more liberal in what it accepts" (July–August 2017).
<https://groups.google.com/a/isocpp.org/forum/#!msg/std-proposals/gQnC5Bkd224/FX7D7OkrBgAJ>.

[N4659]

"Working Draft, Standard for Programming Language C++" (March 2017).
<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/n4659.pdf>.

[P0409R2]

Thomas Köppe. "Allow lambda capture [=, this]" (March 2017).
<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0409r2.html>.